

Introduction

Every automation platform eventually forces you to answer the same question: where do secrets live? n8n is no exception, and it's a question most people answer wrong the first time — not out of carelessness, but because the wrong answer is also the fastest one. You need to hit a Postgres database or call the Slack API, there's a text field right there in the node, and typing the password in gets you unblocked in five seconds.

This matters far beyond n8n. Every interview that touches system design, every code review that includes a config file, every postmortem for a leaked API key comes back to the same principle: secrets and configuration are not the same thing, and they don't belong in the same place. Understanding how n8n separates them — a credentials store with real encryption at rest, and environment variables for everything else — teaches a pattern you'll reuse in Kubernetes secrets, `.env` files, Vault, AWS Secrets Manager, and any other system you touch for the rest of your career.

Problem Overview

n8n workflows are exported as JSON. That's a feature — it's how you version-control automations, share them with teammates, or publish them publicly on GitHub. But a JSON file is just text, and text is only as safe as what you put into it.

If you type a password, API token, or connection string directly into a node's configuration field, that value gets serialized into the exported JSON in plain text. Anyone who opens that file — a teammate, a stranger who forks your public repo, a CI log that happens to print the workflow — can read it directly. Nothing about the export process hides or masks a value that was never treated as a secret in the first place.

The fix isn't a devops tool you have to bolt on. n8n already ships with a dedicated credentials store designed exactly for this. The problem is entirely about habits: most people don't reach for it because the plain text field is right there and works immediately.

Example

Say you have five separate workflows that each need to query the same Postgres database.

The shortcut: paste `postgres://user:hunter2@db.internal:5432/prod` directly into the connection field of each of the five HTTP or database nodes.

What that produces: five node configs with the same password, five separate JSON exports containing that password, and five places a teammate could screenshot, commit, or accidentally paste into Slack.

What happens when the password rotates: you now have to open five workflows, find the field in each, and manually replace the value — and if you miss one, it silently starts failing with an authentication error that looks unrelated to the actual cause.

Compare that to storing it once as a credential named `prod-postgres`. All five workflows reference that one entry. Rotate the password in one place, and every workflow that points to it picks up the new value automatically the next time it runs.

Intuition

Think about how you'd design this yourself, from scratch. You know two things going in: workflows need to be exportable and shareable, and some of the values inside them are sensitive.

The moment you accept both constraints, you arrive at the same conclusion n8n did: the workflow file and the secret value have to be stored separately, and the workflow should only hold a *reference* to the secret, never the secret itself. That's the same shape as a hotel safe — you don't carry the combination around with you, you just use the safe when you need what's inside it. The node is the guest; the credentials store is the safe.

The second piece of intuition is subtler: not every value that changes between environments is a secret. A staging API URL isn't sensitive, but it still shouldn't be hardcoded, because it's different in dev, staging, and production. That's a configuration problem, not a security problem — which is why n8n gives you a second mechanism, environment variables, instead of forcing everything through the encrypted vault. Mixing the two up is itself a design smell: encrypting a URL that doesn't need it, or treating a database password as "just config," both point to a misunderstanding of what the value actually is.

Brute Force Solution

Idea: hardcode every credential and every environment-specific value directly into each node that needs it.

Algorithm: 1. Open the node. 2. Type the value into the relevant field. 3. Repeat for every node, in every workflow, that needs the same value.

Advantages: - Zero setup — no credential entry, no environment configuration. - Works immediately for a single, throwaway workflow.

Disadvantages: - Every export leaks the value in plain text. - Updating a rotated secret means finding and editing every occurrence by hand. - No way to share a workflow with a teammate without also sharing the secret. - Deploying to a new environment means editing the workflow itself instead of just changing configuration.

Complexity: update cost is $O(n)$ in the number of places the value was typed, and leak risk is $O(n)$ in the number of exported copies that exist. Both scale in exactly the direction you don't want.

Optimal Solution

n8n splits the problem into two stores, matched to two different kinds of values:

Value type	Example	Where it lives	Protection
Secret	Slack token, DB password, API key	Credentials store	Encrypted at rest with an instance-level key
Environment-specific config	Staging URL, feature flag, retry count	Environment variables	Not encrypted, but never typed into the node

How the credentials store works: 1. You create a credential once (e.g., `prod-postgres`) and enter the actual value a single time. 2. n8n encrypts that value using an encryption key that lives on the n8n instance itself — not inside the workflow file. 3. Every node that needs that credential references it by name. The node stores the reference, never the raw value. 4. When the workflow runs, n8n decrypts the credential in memory just long enough to use it. 5. When the workflow is exported, the JSON contains the encrypted blob (or in most cases just the reference), which is unreadable without the instance's encryption key.

That last point is the actual security guarantee: if a workflow JSON leaks — a public fork, a misconfigured repo, a shared screenshot — the credential inside it is scrambled without the key that only lives on your instance. A stolen JSON file is a locked safe with no key: heavy, but useless.

How environment variables work: Instead of typing a staging URL into a node, the node references an environment variable, e.g. `{{ $env.API_BASE_URL }}`. That variable is set once per n8n instance. Deploy the same workflow to production, set a different value for that variable on the production instance, and the node automatically resolves to the right address — no editing the workflow itself.

Sharing without leaking: Credentials can be shared with a teammate directly from the vault. They get to use the credential in their own nodes, but they never see the underlying token or password — only n8n does.

Python Code

n8n's credential storage is implemented for you, but the same pattern — an encrypted store keyed by a value that never travels with the data — is worth being able to write from scratch. Here's a minimal, correct version of the same idea in Python, using the standard library's `cryptography` -adjacent approach via `hashlib` / `hmac` replaced with a proper AEAD cipher:

```
import os
import json
from cryptography.fernet import Fernet

class CredentialStore:
    """Encrypts secrets at rest using a key that never leaves the host."""

    def __init__(self, key_path: str = "instance.key"):
        self.key_path = key_path
        self._fernet = Fernet(self._load_or_create_key())

    def _load_or_create_key(self) -> bytes:
        if os.path.exists(self.key_path):
            with open(self.key_path, "rb") as f:
                return f.read()
        key = Fernet.generate_key()
        with open(self.key_path, "wb") as f:
            f.write(key)
        return key

    def save(self, name: str, value: str) -> bytes:
        return self._fernet.encrypt(value.encode())

    def load(self, encrypted: bytes) -> str:
        return self._fernet.decrypt(encrypted).decode()

    def export_workflow(node_config: dict, credential_ref: str) -> str:
        """A workflow export only ever contains a reference, never a raw value."""
        safe_config = {**node_config, "credential": credential_ref}
        return json.dumps(safe_config)
```

Code Walkthrough

- `CredentialStore.__init__` loads an existing key from disk or generates a new one — this key is the equivalent of n8n's instance-level encryption key, and it deliberately lives outside any workflow data.
- `_load_or_create_key` keeps the key on the host filesystem, never inside anything that gets exported or version-controlled.
- `save` encrypts a raw secret using `Fernet`, an authenticated encryption scheme from the `cryptography` library — this models what happens the moment you type a value into n8n's credential form.
- `load` reverses that, but only works if you have the same key file — model this as "the safe, without the key, is just a heavy box."
- `export_workflow` shows the actual discipline: the function that produces exportable JSON only ever receives a `credential_ref` (a name), never the decrypted secret. This mirrors n8n nodes storing a reference instead of a value.

Dry Run

1. First run: `instance.key` doesn't exist, so `CredentialStore` generates one and writes it to disk.
2. `store.save("slack-token", "xoxb-real-value")` returns an encrypted blob, e.g. `b'gAAAAA...'`.
3. That blob (or in n8n's real implementation, just a reference name) is what gets written into any exported workflow JSON.
4. If someone copies the exported JSON to a different machine without `instance.key`, `store.load(blob)` raises an `InvalidToken` error — the data is present but permanently unreadable.
5. On the original machine, with the key intact, `store.load(blob)` correctly returns `"xoxb-real-value"`.

This is exactly the n8n migration gotcha: moving workflows and credentials without moving the encryption key turns every stored credential into permanently unreadable garbage — not leaked, just useless.

Complexity Analysis

- **Time:** encryption and decryption are $O(1)$ relative to the number of workflows using the credential, because the store operates on a single value regardless of how many nodes reference it.

- **Space:** $O(1)$ additional storage per unique credential — one encrypted entry, no matter how many workflows point to it.
- **Update cost:** rotating a credential is $O(1)$ — one update in the store — compared to $O(n)$ for the brute-force approach, where n is the number of places the value was hardcoded.

These are correct because the entire design goal is decoupling the number of *consumers* of a secret from the number of *copies* of that secret.

Alternative Solutions

A common alternative outside n8n is a dedicated secrets manager (HashiCorp Vault, AWS Secrets Manager) that n8n calls out to at runtime instead of storing credentials locally. This is worth it once you have multiple services beyond n8n that need to share secrets, centralized audit logging, or automatic rotation policies. For a single n8n instance, the built-in credentials store covers the same core guarantee — encryption at rest, reference-not-value in exports — with none of the operational overhead of running a separate secrets service.

Edge Cases

- **Empty or missing credential:** a node referencing a credential that was deleted fails at execution time with a clear "credential not found" error rather than silently using a stale value.
- **Duplicate credentials:** creating the same Postgres login twice as two separate credential entries defeats the "update once" benefit — always check if a credential already exists before creating a new one.
- **Minimum case — a single workflow:** even with only one consumer, storing the value as a credential still protects against export leaks; the win isn't purely about reuse.
- **Maximum case — instance migration:** moving to a new server without migrating the encryption key turns every credential into unreadable garbage, as covered above.
- **Non-secret values stored as credentials:** encrypting a staging URL "just to be safe" adds friction without benefit — it belongs in environment variables instead.

Common Mistakes

1. Typing a password or token directly into a node field because it's faster in the moment.
2. Assuming environment variables are automatically secrets — they aren't encrypted, they're just externalized configuration.
3. Hardcoding the same credential into multiple workflows instead of creating one shared entry.

4. Forgetting to migrate the encryption key when moving or backing up an n8n instance.
5. Sharing a credential with a teammate by exporting the workflow JSON rather than sharing it through the vault.
6. Storing non-sensitive, environment-specific values (URLs, flags, retry counts) inside the encrypted credentials store instead of using environment variables.

Interview Questions

- How would you design a system that lets configuration differ across environments without leaking secrets in version control?
- What's the difference between encryption at rest and encryption in transit, and where does each apply in a tool like n8n?
- How would you handle key rotation for an encrypted credentials store without breaking existing workflows?
- What happens to encrypted data if the encryption key is lost — is that a security failure or an availability failure?
- How would you audit who has access to which credentials in a multi-user automation platform?

Similar Problems

- **Secrets vs. config separation** in Kubernetes (`Secret` objects vs. `ConfigMap`) — the same two-store pattern at the orchestration layer.
- **Environment variable management** in twelve-factor apps — config that varies by deploy, kept out of code.
- **Symmetric encryption implementation problems** on LeetCode-adjacent security tracks — implementing or reasoning about AES/Fernet-style encrypt/decrypt flows.
- **Key management design questions** in system design interviews — how a service protects a master key that unlocks everything else.

Key Takeaways

A public leak only happens if a secret was typed somewhere it shouldn't have been. n8n gives you two separate mechanisms on purpose: the credentials store for anything that needs protecting, encrypted with a key that never leaves your instance, and environment variables for anything that just needs to differ by environment. Reference values instead of retyping them, and you get both security and a single place to update when things change. That single habit — asking "does this need protecting, or does this need to vary by environment?" before you type a value anywhere — is the whole lesson.

Watch the Video

For the full walkthrough with the actual n8n interface, credential setup, and environment variable configuration in action, watch the video here: {LINK}

About the Series

This lesson is part of *Fun with Learning Technology*, a series built for developers who want to actually understand the tools they use — not just click through a tutorial once and forget it. Each lesson takes one concept in n8n, automation, or backend engineering and breaks it down the way you'd explain it to a teammate: the mistake people make, why it happens, and the mental model that prevents it next time.

Call To Action

If this saved you from a debugging session down the line, do three things: like the video on YouTube so more people see it, subscribe to the channel for the rest of the n8n series, and subscribe to the Substack for the written breakdowns like this one. Drop a comment with the credential mistake that got you, and share this with a teammate who's still typing passwords into nodes.

Quick Review

n8n credentials: encrypted at rest or plaintext DB rows?

Encrypted at rest — n8n encrypts credential data using an encryption key before storing.

Why store secrets in n8n Credentials instead of env vars in a node?

Reuse across workflows without duplication, plus encryption; env vars are still visible to anyone who can read the process/config.

Tell: workflow needs same API key in 5 places — what's the fix?

Create one Credential, attach it to each node — avoids duplicated, unrotatable, leak-prone copies.

Common bug: pulling a value from process.env inside a Function node

Exposes the secret in node output/logs; use a Credential so the value is used but never displayed.

Is every environment variable a secret needing a Credential?

No — config values like a base URL or feature flag don't need encryption; only real secrets (keys, tokens, passwords) do.

Interview follow-up: how would a leaked credential happen despite encryption at rest?

Via logging/printing the resolved value at runtime, or overly broad workflow-sharing permissions exposing the credential reference.

n8n Lesson 25: Environment variables and credentials management (storing secrets securely, reusing credentials across workflows) — free Python cheat sheet